ENGINEERING DOCUMENTATION PORTFOLIO · VOLUME 11

MCP, Tools, and Integrations

MCP discovery, business systems, email, Drive, and external actions.

Eurskem AI · Agentic Workflow Platform

Generated 2026-08-24

Code revision 8bd16ca4a2a3

Derived from the live repository

> **Source of truth.** Inventories and symbol tables are generated from the current checkout. Narrative explains observed structures and does not replace executable contracts.

Integration model

MCP servers expose discoverable tools and schemas. The Builder reads live manifests instead of hardcoding every capability. Integration nodes execute through configured services; writes are subject to identity, policy, idempotency, and audit controls. Adapters cover email, Dynamics, business records, Drive, web search, databases, and external HTTP actions.

File	Responsibility
`app/api/candidates.py`	Candidates viewer API: list a run's acquired evidence PDFs with open links, plus the raw discovery/Deep Research/prior-project/official-database candidates (name + URL) recorded for the run, since ScholarlyCandidateDiscoveryAgent, BoundedDeepResearchAgent, PriorProjectRetrieverAgent, and StructuredDatasetRetrieverAgent all produce either the same CandidateSource shape or a reconstructible exact-query URL regardless of source type.
`app/api/email_oauth.py`	Email OAuth connect flow — Outlook (Microsoft Graph) and Gmail.
`app/api/integration_oauth.py`	Integration OAuth connect flow — Google Drive and OneDrive.
`app/integrations/__init__.py`	Python module.
`app/integrations/email/__init__.py`	Email integration — one capability, provider adapters underneath.
`app/integrations/email/base.py`	Provider-neutral email contract.
`app/integrations/email/connections_store.py`	Durable, non-secret record of an OAuth-established email connection.
`app/integrations/email/gmail.py`	Gmail adapter — Gmail API v1 over the shared httpx client.
`app/integrations/email/memory.py`	In-memory email adapter.
`app/integrations/email/msgraph.py`	Microsoft Graph adapter — Outlook / Microsoft 365 mail.
`app/integrations/email/oauth.py`	OAuth authorization-code flow for Microsoft Graph and Gmail.
`app/integrations/email/service.py`	EmailService — connection resolution, idempotency, and the operation switch.
`app/integrations/email/token_vault.py`	Encrypted storage for OAuth-issued email tokens.
`app/integrations/external_action.py`	ExternalActionService — the one place an outbound REST/webhook call goes through.
`app/integrations/files/__init__.py`	Cloud file integrations — one capability, provider adapters underneath.
`app/integrations/files/base.py`	Provider-neutral cloud-file contract.
`app/integrations/files/connections_store.py`	Durable, non-secret record of an OAuth-established Integration connection.
`app/integrations/files/google_drive.py`	Google Drive adapter — Drive API v3 over the shared httpx client.
`app/integrations/files/oauth.py`	OAuth authorization-code flow for Google Drive and OneDrive.
`app/integrations/files/onedrive.py`	OneDrive adapter — Microsoft Graph over the shared httpx client.
`app/integrations/files/service.py`	IntegrationService — connection resolution, refresh, and the operation switch.
`app/integrations/files/token_vault.py`	Encrypted storage for OAuth-issued Google Drive / OneDrive tokens.

File	Responsibility
app/integrations/operations.py	Durable record of attempted external side effects.
app/integrations/url_guard.py	URL safety guard for ExternalActionAgent outbound calls (SSRF protection).
app/mcp/business_records/__init__.py	Real, live MySQL-backed business-records MCP server.
app/mcp/business_records/db.py	Connection helper for the business-records MySQL database.
app/mcp/business_records/handlers.py	Real, parameterized-SQL handlers for the business-records MCP server.
app/mcp/business_records/schema.sql	-- Business Records MySQL schema.
app/mcp/business_records/seed.py	Load both fixture files into the real MySQL database.
app/mcp/business_records/server.py	Business Records MCP server — a real, persistent MySQL-backed connector.
app/mcp/business_records/sql_guard.py	Defense-in-depth for the query_readonly MCP tool.
app/mcp/business_records/tools.py	Tool definitions for the business-records MySQL server.
app/mcp/client.py	MCP client wrapper — multi-server.
app/mcp/connections.py	Built-in MCP connections, and how the integration service is assembled.
app/mcp/d365_finance/__init__.py	Fixture-backed mock for the d365-finance-scm-mcp business-tool layer.
app/mcp/d365_finance/fixtures.json	{ "customers": [{ "customerid": "f1a2c3d4-0000-4000-8000-000000000001", "customer_account": "C0001", "name": "Meridian Process Systems", "data_area_id": "usmf", "sales_region": "EMEA", "key_account": true, "account_owner_name": "Elena Cross", "territory_sales_owner_name": "", "sales_engineer_name":
app/mcp/d365_finance/handlers.py	Fixture-backed handlers for the d365-finance-scm-mcp business-tool layer.
app/mcp/d365_finance/server.py	Fixture-backed mock for the Dynamics 365 Finance & Operations MCP server.
app/mcp/d365_finance/tools.py	Atomic read tools for the fixture-backed D365 Finance & SCM connector.
app/mcp/dynamics/__init__.py	Python module.
app/mcp/dynamics/client.py	Dataverse Web API client and its fixture-backed twin.
app/mcp/dynamics/fixtures.json	{ "accounts": [{ "accountid": "a1b2c3d4-0000-4000-8000-000000000001", "name": "ABC Chemicals B.V.", "accountnumber": "ACC-1042", "industrycode": "Chemicals", "address1_city": "Rotterdam", "address1_country": "Netherlands", "telephone1": "+31 10 555 0142", "websiteurl": "https://abc-chemicals.exampl
app/mcp/dynamics/handlers.py	Tool implementations: business question in, typed record out.
app/mcp/dynamics/odata.py	Safe construction of Dataverse Web API queries.
app/mcp/dynamics/server.py	The Dynamics 365 MCP server.
app/mcp/dynamics/tools.py	The Dynamics 365 CRM tool vocabulary.
app/mcp/paper_search_server.py	Credential-aware launcher for the pinned paper-search-mcp server.
app/mcp/policy.py	What may be called, by whom, and whether a person has to see it first.
app/mcp/registry.py	MCP servers as first-class connections.
app/mcp/results.py	Turn an MCP tool result into a typed workflow output.
app/mcp/server.py	MCP server: exposes hybrid retrieval as protocol-level tools.
app/mcp/service.py	The MCP integration service: discovery, policy, execution, audit.
app/tools/database_lookup.py	Bounded clients for documented public database APIs.
ui/src/modes/studio/RunDialog.tsx	

File	Responsibility
	Fields declared directly on a Start node in `input_form` mode — read from the workflow YAML itself rather than the lossy, collapsed `inputs:` block (every non-file Start field flattens to `type: "json"` there), so the rich widgets (email/currency/date-range/ repeating-group/conditional visibility) ar
`ui/src/modes/studio/builder/CloudFileBrowser.tsx`	A live folder/search browser for a connected Google Drive or OneDrive account, embedded in the Integration node's configuration panel.
`ui/src/modes/studio/builder/EmailConfig.tsx`	The Email capability's configuration.
`ui/src/modes/studio/builder/ExternalActionConfig.tsx`	Configuring a call to a system this deployment has no MCP connection for — a specific URL the author already knows.
`ui/src/modes/studio/builder/IntegrationConfig.test.tsx`	React/TypeScript module.
`ui/src/modes/studio/builder/IntegrationConfig.tsx`	The Integration capability's configuration.
`ui/src/modes/studio/builder/MCPToolConfig.tsx`	Re-exported for existing call sites (`EmailConfig.tsx`, `ExternalActionConfig.tsx`,
`ui/src/modes/studio/builder/MCPToolPicker.tsx`	Add an MCP tool by searching what it does, across every connected system at once — instead of picking a server first and only then discovering what it can do.
`ui/src/modes/studio/builder/OperationBadge.tsx`	Shared safety-classification badge — one visual language for every node that touches something outside the platform, not just MCP tools.
`ui/src/modes/studio/builder/TemplateTextField.tsx`	A free-text field (a prompt, an email body, a document template) that can embed references to earlier steps' data — shown as chips, not raw `{{node.field}}` syntax, while still serializing to exactly that string underneath.

Method	Path	Auth	Handler	Purpose
POST	`/api/builder/assist/rules`	consultant	`app/api/builder.py:1032` assist_rules	Turn a described business rule into deterministic configuration (§36).
POST	`/api/builder/assist/schema`	consultant	`app/api/builder.py:913` assist_schema	Propose a structured-output schema from a description (§37).
GET	`/api/builder/email/connect/{provider}`	consultant	`app/api/email_oauth.py:146` connect	Starts the authorization-code flow: stash a one-time state (+ PKCE verifier for Gmail), redirect the user's browser to the provider.
GET	`/api/builder/email/connections`	consultant	`app/api/builder.py:1300` email_connections	Configured mailboxes for the Email node's connection picker.
DELETE	`/api/builder/email/connections/{connection_id}`	consultant	`app/api/email_oauth.py:303` disconnect	Compute the disconnect.
PATCH	`/api/builder/email/connections/{connection_id}`	consultant	`app/api/email_oauth.py:274` update_connection	The only lever to actually let a connection send — every connection (OAuth-established or static) starts refused; this is the explicit second step, same reasoning as every other write-capable connection in this platform (MCP, F&O).
GET	`/api/builder/email/oauth/callback/{provider}`	public	`app/api/email_oauth.py:178` oauth_callback	The provider redirects the user's browser back here after consent.
GET		consultant		Starts the authorization-code flow: stash a one-time state (+ PKCE verifier for

Method	Path	Auth	Handler	Purpose
	`/api/builder/integrations/connect/{provider}`		`app/api/integration_oauth.py:137` connect	Google), redirect the user's browser to the provider.
GET	`/api/builder/integrations/connections`	consultant	`app/api/integration_oauth.py:249` list_connections	List the connections.
DELETE	`/api/builder/integrations/connections/{connection_id}`	consultant	`app/api/integration_oauth.py:270` disconnect	Compute the disconnect.
GET	`/api/builder/integrations/connections/{connection_id}/download/{file_id}`	consultant	`app/api/integration_oauth.py:359` download_file	Streams one file's real bytes straight to the browser — a plain navigation (an `<a href>` click, not a fetch), authenticated by the same HttpOnly cookie the OAuth popup flow relies on.
GET	`/api/builder/integrations/connections/{connection_id}/files`	consultant	`app/api/integration_oauth.py:295` browse_files	Live folder/search browsing for the node config panel's file picker — not workflow execution, just a thin proxy through the service for whichever operation the presence of `query` selects.
GET	`/api/builder/integrations/connections/{connection_id}/path/{file_id}`	consultant	`app/api/integration_oauth.py:327` browse_path	Breadcrumb chain for the file browser: walk `parent_id` up from `file_id` via repeated lookups, bounded so a cyclic/misbehaving provider response can never hang this request.
GET	`/api/builder/integrations/oauth/callback/{provider}`	public	`app/api/integration_oauth.py:169` oauth_callback	The provider redirects the user's browser back here after consent.
GET	`/api/builder/mcp/servers`	consultant	`app/api/builder.py:1124` mcp_servers	Configured MCP servers for the Builder's connection panel.
GET	`/api/builder/mcp/servers/{server_id}/health`	consultant	`app/api/builder.py:1176` mcp_health	Live connection check, for the panel's Test Connection button.
GET	`/api/builder/mcp/servers/{server_id}/tools`	consultant	`app/api/builder.py:1142` mcp_tools	Discover what a server can do (§5).
POST	`/api/builder/mcp/test-tool`	consultant	`app/api/builder.py:1207` mcp_test_tool	Run one MCP tool with literal arguments (§21).
POST	`/api/builder/node-test`	consultant	`app/api/builder.py:358` node_test	Run one node against sample data (§21).
GET	`/api/builder/operators`	consultant	`app/api/builder.py:112` operators	Which operators apply to which field type, and how to render each one.
POST	`/api/builder/output-contract`	consultant	`app/api/builder.py:203` output_contract	What each step guarantees, as typed dotted paths (§40).
POST	`/api/builder/schema-preview`	consultant	`app/api/builder.py:296` schema_preview	Compile visual schema rows and show what the model will be held to.
POST	`/api/builder/simulate`	consultant	`app/api/builder.py:507` simulate	Run a workflow in memory and return a per-step trace (§23, §24, §47).